Sesion 6: Contenedores, Self-Healing y Contract Testing

Docker / Podman · Serenity BDD + Healenium · Pact (Python y Java) · K8s

Certificacion 5: QA Cloud-Native - Automatizacion Inteligente con IA Generativa

1. Pre-requisitos
2. Puente Dia 5
3. Por que contenedores
4. Docker vs Podman
5. Dockerfile linea a linea
6. Serenity BDD + Screenplay
7. Healenium (self-healing real)
8. Stack completo en contenedores
Ej. Self-Healing
9. Contract testing
10. Pact en Python
11. Pact en Java
Ej. Romper contrato
12. Kubernetes (conceptual)
Cierre

1. Pre-requisitos y estructura del Dia 6

Que necesitas y que hay en cada carpeta

Herramienta	Para que	Verificar
Podman 5.x (o Docker)	Construir y correr todos los contenedores	`podman --version`
Java 17+	Solo si corres Gradle fuera del contenedor	`java -version`
Python 3.11	Solo si corres el consumer Pact Python fuera del contenedor	`python --version`

Estructura del Dia 6

dia6/proyecto/

```
provider-tienda/              # API Node — SUT y provider Pact (Python y Java)
consumer-notificaciones/      # Consumer Pact en Python
provider-verify/              # Verificador Pact en Python
pacts/                        # Contrato generado (compartido entre Python y Java)

tienda-serenity-java/         # Proyecto Java integrador
  app-tienda/                 #   App Tienda contenerizada (Node)
  src/test/java/...           #   Serenity BDD + Screenplay + Cucumber
  src/test/resources/         #   serenity.conf + features
  podman-compose.yml          #   Stack completo (7 servicios) — se levanta con podman compose
```

💡 Los dos proyectos (Python y Java) usan la **misma app Tienda** como sistema bajo prueba. Python = contract testing liviano. Java = E2E + self-healing + contract testing, todo en contenedores.

MAPA DE EJERCICIOS: que carpeta y que comando usar

Cada ejercicio vive en una carpeta concreta y se ejecuta con un comando. Todo corre en contenedores (Podman por defecto; para Docker usa `docker` o `$env:ENGINE="docker"`).

Ejercicio	Carpeta (desde `dia6/proyecto/`)	Comando para ejecutar	Resultado
Ej. 1 Construir y correr la Tienda	`provider-tienda/`	`podman build -t provider-tienda .` `podman run -d --name tienda -p 3000:3000 provider-tienda`	API en `localhost:3000`
Ej. 2 E2E + self-healing (Healenium)	`tienda-serenity-java/` el cambio se hace en `app-tienda/public/index.html`	`podman compose up -d` (levanta el stack) `podman build -t tienda-tests -f Dockerfile.tests .` `podman run --rm --network tienda-serenity-java_default -e HLM_PROXY_URL=http://hlm-proxy:8085/wd/hub -e PROVIDER_BASE_URL=http://app-tienda:3000 tienda-tests gradle --no-daemon --rerun-tasks -DbaseUrl=http://app-tienda:3000 e2e` cambia el `data-testid` en la app → repite el `podman run ... e2e` (sigue verde, Healenium sana). **NO uses `compose run tests` en Windows.**	tests=3, failures=0 · healing_result score=0.964 (registro intermitente en Windows)
Ej. 3a Contract testing — Python	`dia6/proyecto/` (raiz)	3 pasos con `podman run` (consumer → provider → verify). Ver seccion 10.	"1 interaction, 0 failures"
Ej. 3b Contract testing — Java	`tienda-serenity-java/`	`podman run --rm --network tienda-serenity-java_default -v pact-data:/work/build/pacts -e PROVIDER_BASE_URL=http://app-tienda:3000 tienda-tests gradle --no-daemon consumerPact` luego lo mismo con `providerPact` (mismo `-v pact-data`). **NO `compose run tests` en Windows.**	consumerPact + providerPact OK
Ej. 3c Romper/reparar el contrato	Python: `provider-tienda/servidor/server.js` Java: `app-tienda/servidor/server.js`	edita `precio` → `valor`, reconstruye+recrea y re-verifica (Java: con `--rerun-tasks`)	FALLA (1 failure) → revierte → 0 failures

⚠️ **Importante — TODO corre en contenedores.** No necesitas instalar Java, Node, Python, Selenium ni PostgreSQL en tu maquina: cada pieza tiene su `Dockerfile` y el stack Java tiene su `podman-compose.yml`. Solo necesitas Podman (o Docker).

Donde esta cada Dockerfile (todo contenerizado)

- `provider-tienda/Dockerfile` — API Tienda (Node). Provider de Pact Python y Java.
- `consumer-notificaciones/Dockerfile` — consumer Pact Python.
- `provider-verify/Dockerfile` — verificador Pact Python.
- `tienda-serenity-java/app-tienda/Dockerfile` — app Tienda para el stack Java.
- `tienda-serenity-java/podman-compose.yml` — orquesta los 7 servicios del stack Java (app + postgres + healenium + selector-imitator + selenium-hub + chrome + tests).

📘 DOCUMENTACION DE REFERENCIA

Comandos Podman mas usados (chuleta)

Comandos esenciales

```
podman build -t nombre ./carpeta    # construir imagen desde Dockerfile
podman run -d --name c1 -p 3000:3000 nombre   # correr en background
podman ps                           # contenedores corriendo
podman logs -f c1                   # ver logs en vivo
podman exec -it c1 sh               # entrar al contenedor
podman rm -f c1                     # forzar borrado
podman network create mi-red        # red para que contenedores se vean por nombre
podman compose up -d                # levantar stack completo
```

```
podman compose down -v          # apagar y limpiar (incluye volumenes)
podman system prune -f          # limpieza general de lo no usado
```

Ref: docs.podman.io

2. Puente desde el Dia 5

De donde venimos y por que este es el siguiente paso

El recorrido del modulo hasta aqui

Dias 1-2
Datos sinteticos
+ API base

→

Dia 4
E2E + BDD
Playwright Python

→

Dia 5
Nube + IA
Sauce Labs

→

Dia 6
Contenedores
Healenium + Pact

→

Dia 7
DORA + cierre

Que suma el Dia 6 al recorrido

- **Empaquetado reproducible:** la app Tienda en un contenedor que corre igual en tu PC, en CI y en produccion.
- **Self-healing de selectores:** con Serenity + Healenium, un cambio de `data-testid` ya no rompe el test — Healenium encuentra el elemento y reporta la sugerencia.
- **Contract testing:** validamos que la API cumple el contrato que otro servicio espera, sin E2E pesados. Dos implementaciones: Python (liviana, ya conocida) y Java (Pact JVM, produccion-ready).

PARTE A

Docker / Podman: empaquetar la app

El problema del "en mi maquina funciona" y su solucion. Construimos la imagen de la Tienda y la levantamos como contenedor.

Secciones 3-5 · Todo corre de verdad

3. Por que contenedores en QA cloud-native

El problema que resuelven y las 4 ventajas clave

El problema clasico

Una prueba pasa en tu PC pero falla en CI porque la version de Node, el sistema operativo o una variable de entorno cambia. El contenedor empaqueta la app **junto con su entorno exacto** (SO base, runtime, dependencias). La misma imagen corre identica en tu PC, en Azure Pipelines y en produccion.

Reproducible

La imagen es la misma en todos lados. Se acabo el "en mi maquina si". Cualquier agente de CI obtiene el mismo resultado.

Aislado

Cada servicio corre en su caja, con sus dependencias, sin pisarse. Postgres de Healenium no interfiere con Node de la Tienda.

Efimero

Levantas, pruebas, destruyes. Ideal para ambientes de test limpios en cada corrida. `podman compose down -v` borra todo.

Portable a la nube

La misma imagen va a Kubernetes, ECS, Cloud Run... sin cambios. Lo que aprendes hoy se usa en produccion.

4. Docker vs Podman

Dos herramientas, el mismo formato de imagen (OCI). Los Dockerfile son identicos.

Aspecto	Docker	Podman
Demonio	Necesita dockerd corriendo	Sin demonio (daemonless)
Permisos	Suele requerir privilegios	Rootless por defecto (mas seguro)
Comandos	`docker build/run/...`	`podman build/run/...` (identicos)
Formato imagen	OCI	OCI (100% compatible con Docker)
Compose	`docker compose`	`podman compose`

> 💡 Usamos Podman porque no exige demonio ni admin. **Todo lo que ves aplica tal cual a Docker**: cambia `podman` por `docker` y funciona igual.

5. Dockerfile de la app Tienda — linea a linea

Construir la caja: capas, cache, datos deterministas y healthcheck

provider-tienda/Dockerfile

```
FROM node:20-alpine            # imagen base minima (~50 MB)
WORKDIR /app

# 1) package.json ANTES del codigo: si solo cambia el codigo,
#    Podman reutiliza la capa de dependencias → build mucho mas rapido
COPY package*.json ./
RUN npm install --omit=dev --ignore-scripts

# 2) Copiar el resto (servidor/, datos/, public/)
COPY . .

# 3) Generar datos/db.json con Faker (semilla fija → siempre el mismo producto 1)
RUN node datos/generar-datos.js

ENV PORT=3000
EXPOSE 3000

# Healthcheck: el orquestador sabe cuando la app esta lista
HEALTHCHECK --interval=15s --timeout=3s --start-period=5s --retries=3 \
  CMD wget -qO- http://localhost:3000/api/health || exit 1

CMD ["node", "servidor/server.js"]
```

Las 3 ideas clave de este Dockerfile

- **Orden de capas:** `package.json` + `npm install` antes del codigo. Si el codigo cambia pero no las dependencias, Docker/Podman reutiliza la capa de `npm install` (mucho mas rapido).
- **Datos deterministas:** Faker con semilla `12345`. El producto 1 es SIEMPRE `Tasty Concrete Chair, hogar, $106.25`. Esto estabiliza las pruebas E2E y el contrato Pact.
- **Healthcheck:** el contenedor dice "ya estoy listo". El orquestador espera esto antes de correr las pruebas — evita el "la prueba empezo antes de que la app arrancara". *Ojo con Podman:* construye con `podman build --format docker` para que el HEALTHCHECK no se ignore (en formato OCI, el default de Podman, se descarta con un warning).

🖍️ **EJERCICIO PRACTICO**

Ejercicio 1 — Construir y correr el contenedor de la Tienda

📁 Carpeta: `dia6/proyecto/provider-tienda/`

1. **Construir la imagen:** `podman build --format docker -t provider-tienda .`
   Verifica: al final debe decir `Successfully tagged localhost/provider-tienda:latest`. Confirma con `podman images` → la imagen `provider-tienda` aparece en la lista con su tamaño y fecha.
   ¿Por que `--format docker`? Podman construye en formato OCI por defecto, y en OCI la instruccion `HEALTHCHECK` del Dockerfile se IGNORA (sale un warning). Con `--format docker` el healthcheck si funciona y veras `(healthy)` en el paso 4.

2. **Correr el contenedor:** `podman run -d --name tienda -p 3000:3000 provider-tienda`
   Verifica: el comando imprime un ID largo (el del contenedor). Confirma con `podman ps` → `tienda` aparece con STATUS `Up X seconds`.

3. **¿Arrancó bien por dentro?** `podman logs tienda`
   Verifica: debe verse el banner "App Tienda corriendo" y "Productos: 24". Si hay un error de Node, aparece aqui.

4. **¿Esta sana la app?** espera ~10s y mira `podman ps` → el STATUS pasa de `(starting)` a `Up X seconds (healthy)` (solo si construiste con `--format docker`). La prueba definitiva, que SIEMPRE funciona: `curl http://localhost:3000/api/health` → `{"estado":"ok"...}`

5. **Probar la API:** `curl http://localhost:3000/api/productos/1` → observa la FORMA (id, nombre, categoria, precio, stock). Esta es la forma que el contrato Pact validara despues.

6. **Limpiar:** `podman rm -f tienda`. Confirma con `podman ps` → ya no aparece. Efimero: levantamos, usamos, tiramos.

Como saber en que estado esta cada cosa

Quiero saber...	Comando	Que buscar
¿La imagen se construyo?	`podman images`	la imagen `provider-tienda` en la lista (+ mensaje `Successfully tagged` al build)
¿El contenedor esta corriendo?	`podman ps`	STATUS = `Up ...`. Si no aparece, esta detenido o fallo
¿Fallo al arrancar?	`podman ps -a`	si STATUS = `Exited (1)` hubo un error; velo con `podman logs`
¿La app esta lista/sana?	`podman ps`	STATUS muestra `(healthy)` cuando el HEALTHCHECK pasa
¿Que paso por dentro?	`podman logs tienda`	el log de Node (banner de arranque o el error)
¿Responde la API?	`curl .../api/health`	`{"estado":"ok"}`
¿Cuanto ocupa / que capas tiene?	`podman history provider-tienda`	las capas del Dockerfile y su tamaño

💡 Regla rapida para leer `podman ps`: la columna **STATUS** es tu semaforo. `Up (healthy)` = todo bien · `Up (starting)` = espera unos segundos · `Up (unhealthy)` o `Exited` = revisa `podman logs`.

⚠️ **Si el build falla:** Podman se detiene en el paso (STEP x/y) que fallo y muestra el error. Los mas comunes: sin conexion para bajar `node:20-alpine`, o un error en `npm install`. Corrige y vuelve a `podman build` — Podman reutiliza las capas que ya funcionaron (cache), asi que el segundo intento es mas rapido.

PARTE B

Serenity BDD + Screenplay + Healenium (self-healing real)

El mismo patron Screenplay que ya conoces en Python, ahora en Java con Serenity. El motor cambia a Selenium, lo que habilita **Healenium**: self-healing de selectores de verdad, no conceptual.

Secciones 6-8 · Todo corre de verdad (tests=3, failures=0 verificado)

6. Serenity BDD + Screenplay en Java

El mismo patron, otro lenguaje. Por que Java habilita Healenium.

Por que Java para self-healing

Healenium intercepta el **WebDriver** de Selenium. Playwright (nuestro stack Python) no usa WebDriver: habla directo por CDP. Por eso Healenium no tiene version para Playwright — es una decision arquitectonica, no una limitacion del curso. Para la demo real de self-healing necesitamos un motor Selenium, y Serenity BDD en Java es el stack mas maduro con ese motor.

Screenplay en Java = el mismo patron que ya conoces

Concepto	Python (Playwright, Dias 4-5)	Java (Serenity, Dia 6)
Actor	`Actor.llamado("Ana")`	`OnStage.theActorCalled("Ana")`
Task	`actor.intenta(AbrirElCatalogo())`	`actor.attemptsTo(AbrirLaHome.laHome())`
Question	`actor.pregunta(ContadorCarrito.valor())`	`actor.asksFor(ContadorCarrito.valor())`
Target/localizador	`page.locator("[data-testid=...]")`	`Target.the("...").locatedBy("[data-testid=...]")`
Motor	Playwright (CDP)	Selenium WebDriver → **habilita Healenium**

El feature se lee igual (Gherkin en espanol)

catalogo.feature

```
# language: es
@e2e @catalogo @healenium
Caracteristica: Catalogo de productos
  Antecedentes:
    Dado que Ana abre la home de la tienda

  Escenario: Filtrar el catalogo por la categoria hogar
    Cuando Ana filtra el catalogo por la categoria "hogar"
    Entonces Ana solo ve productos de la categoria "hogar"
    Y la cantidad de productos visibles es mayor que cero
```

El negocio lee lo mismo que en el Dia 4. La diferencia esta debajo: los steps llaman Tasks y Questions de Screenplay en Java, y el driver tiene Healenium interpuesto.

Mensaje clave para el alumno

Aprendiste un patron, no una herramienta. Screenplay viaja entre lenguajes y motores. El dia que tu empresa use otro stack, el razonamiento (Actor → Task → Question) es el mismo.

7. Healenium: self-healing de selectores (demo REAL)

Que es, como funciona, y el flujo verificado con Podman

Que es Healenium

Healenium es una libreria open source que da **self-healing** a pruebas con Selenium. Cuando un localizador deja de encontrar su elemento (porque el DOM cambio), en vez de fallar de inmediato, Healenium busca el elemento **mas parecido** al original (algoritmo de similitud sobre el arbol DOM guardado en PostgreSQL), continua la prueba y **reporta** que localizador sano y cual uso en su lugar.



Como se engancha (sin libreria en el codigo: via hlm-proxy)

El enfoque oficial de Healenium para Serenity NO usa una libreria en el proyecto. Serenity apunta su WebDriver remoto al servicio **hlm-proxy**, que se interpone entre Serenity y el Selenium Grid. Una sola linea en `serenity.conf` :

serenity.conf (resumido)

```
webdriver {
  driver = remote
  # build.gradle fija remote.url = http://hlm-proxy:8085/wd/hub
  remote.driver = chrome
}
```

El proxy recibe cada busqueda; si el selector falla, consulta al backend de Healenium, encuentra el elemento mas parecido y continua. Serenity trabaja igual que siempre: no sabe que hay un "sanador" debajo. (Para correr SIN healing y comparar, se apunta `remote.url` directo al grid.)

Cuando usar Healenium (y cuando no)

Encaja bien

Suites grandes con selectores volatiles. Red de seguridad mientras se estabilizan los `data-testid` . Equipos que quieren reducir mantenimiento por cambios cosmeticos de UI.

Evitar / con cuidado

Suites nuevas (mejor invertir en buenos selectores). Pruebas criticas donde un "falso verde" es inaceptable. Sin proceso de revision de los healings reportados.

Resultado verificado con Podman 5.7.1 (ciclo completo)

- **Corrida sana:** E2E Serenity 3 pasaron, 0 fallaron. Healenium aprendio 7 selectores (tabla `healenium.selector`).
- **Tras cambiar el `data-testid` en la app:** el E2E SIGUE en verde (3/0). Healenium sano el selector: tabla `healenium.healing_result` → localizador `select#filtro-categoria`, score **0.964**, `success_healing=true`.
- Confirmado: el self-healing ocurre de verdad, no solo el aprendizaje.

8. Stack completo en contenedores

Que habla con que — arquitectura del proyecto Java

Red de contenedores (podman-compose.yml)

```
tests (Gradle+Java, Serenity)
    | WebDriver remoto
    ↓
hlm-proxy:8085  ──────→  selenium-hub ──→ chrome (navegador real)
    |  si un selector falla,                    | navega la UI
    |  pide ayuda a healenium                   ↓
    ↓                                  app-tienda:3000 (Node + front + API)
healenium:7878 (backend)
    | guarda / consulta huellas          selector-imitator:8000
    ↓                                    (sugiere el nuevo selector)
postgres-hlm:5432 (huellas DOM)
```

Servicio	Imagen	Rol	Puerto
`app-tienda`	build local (Node 20)	SUT: front + API; tambien provider Pact	3000
`postgres-hlm`	postgres:15-alpine	Base de datos de Healenium (huellas)	5432
`selector-imitator`	hlm-selector-imitator:1.6	ML: sugiere el nuevo localizador	8000
`healenium`	hlm-backend:3.5.1	Backend de self-healing (Spring Boot). Flag `FIND_ELEMENTS_AUTO_HEALING=true`	7878
`hlm-proxy`	hlm-proxy:2.2.1	**Proxy donde Serenity conecta.** Aqui ocurre el healing	8085
`selenium-hub`	selenium/hub:4.25.0	Selenium Grid (hub)	4444
`chrome`	selenium/node-chromium:4.25.0	Navegador real headless	—
`tests`	gradle:8.10.2-jdk21	Corre E2E + Pact Java	—

> 💡 Este mismo grafo corre igual en tu PC, en CI y en la nube. Nadie instala Java, Selenium ni PostgreSQL en su maquina: todo vive en contenedores. **Esa es la promesa cloud-native.**

> ⚠️ **Entiende el escenario ANTES de hacerlo:** el self-healing NO se prueba cambiando el selector en el TEST. Se prueba cambiando el **HTML de la app** (como haria un desarrollador que renombra un `data-testid`). El test sigue buscando el selector viejo; Healenium re-encuentra el elemento por **similitud del arbol DOM** (el `<select>` sigue ahi, solo cambio su atributo) y sana la prueba. Si cambiaras el selector del test a uno nuevo, Healenium no tendria huella de ese selector y NO podria sanar.

✏️ **EJERCICIO PRACTICO**

Ejercicio 2 — Self-healing en accion (romper el HTML de la app, no el test)

📂 Carpeta: `dia6/proyecto/tienda-serenity-java/` · el cambio se hace en `app-tienda/public/index.html` (la APP), NO en el test

Cada paso trae su **comando de verificacion** y la **salida esperada**. No avances al siguiente paso si la verificacion no coincide: esa es la forma de saber que vas bien.

PASO 0 — Reinicio limpio (ejecutalo SIEMPRE antes de empezar)

```
# Desde dia6/proyecto/tienda-serenity-java/
# 1) Baja todo y borra el volumen de PostgreSQL (huellas viejas)
podman compose down -v

# 2) Asegura el HTML de la app en su estado original (data-testid="filtro-categoria")
#    Si lo editaste antes, revierte selector-categoria -> filtro-categoria.
```

```
#    Verifica:
Select-String -Path app-tienda/public/index.html -Pattern 'data-testid="filtro-categoria"'

# 3) RECONSTRUYE la imagen de la app desde el HTML limpio (clave: evita servir un DOM roto de una corrida
anterior)
podman build -t tienda-serenity-java_app-tienda ./app-tienda

# 4) Confirma que la imagen trae el HTML limpio
podman run --rm tienda-serenity-java_app-tienda cat /app/public/index.html | Select-String "filtro-categoria"

# 5) Borra screenshots de corridas previas (opcional)
Remove-Item -Recurse -Force .\screenshots -ErrorAction SilentlyContinue
```

⚠️ **Por que el Paso 0 importa:** si en una corrida anterior dejaste el `data-testid` roto y **construiste la imagen** con ese HTML, un simple `podman compose up` reusa esa imagen rota: la app sirve el DOM roto y la "corrida sana" falla con `NoSuchElementException`. Ademas, sin `down -v`, las huellas viejas de PostgreSQL pueden quedar incompletas. El Paso 0 garantiza empezar desde cero: imagen limpia + base de datos vacia.

1. **Levanta el stack completo:** `podman compose up -d` (7 servicios: app-tienda, postgres-hlm, selector-imitator, healenium, hlm-proxy, selenium-hub, chrome).

   ✓*Verifica (stack arriba):* `podman compose ps` → los 7 servicios en estado `Up` / `running`.

   ✓*Verifica (grid listo):* `curl http://localhost:4444/status` → responde JSON con `"ready":true`.

   ✓*Verifica (app responde):* `curl http://localhost:3000/api/health` → `{"estado":"ok","servicio":"app-tienda",...}` (HTTP 200).

   ✓*Verifica (healing plural activo):* `podman compose exec healenium env | Select-String FIND_ELEMENTS_AUTO_HEALING` → `FIND_ELEMENTS_AUTO_HEALING=true`. Sin esto, el self-healing NO ocurre con Serenity.

   ✕ Si algun servicio aparece `Exited`: `podman compose logs <servicio>` para ver el error antes de seguir.

2. **Corrida sana (Healenium APRENDE):** corre el E2E con `podman run` directo sobre la imagen `tienda-tests` (NO `podman compose run tests`).

   ⚠️ **Por que NO `podman compose run tests` en Windows:** el servicio `tests` por compose, con bind mount `.:/work`, falla en Podman/WSL (lock de Gradle y el E2E cae con *"no matching element found by [grid de productos]"* aunque la app esta bien). Por eso el servicio `tests` esta marcado `profiles: ["manual"]` (no sube con `compose up`) y los tests corren con `podman run` sobre la imagen `tienda-tests` (codigo DENTRO de la imagen, sin bind mount). **Verificado:** asi pasa 3/3; por compose, falla.

   E2E — corrida sana (desde tienda-serenity-java/)

   ```
   # (una vez) construir la imagen de tests con el codigo dentro
   podman build -t tienda-tests -f Dockerfile.tests .

   # corrida E2E (self-healing via hlm-proxy)
   podman run --rm --network tienda-serenity-java_default `
     -e HLM_PROXY_URL=http://hlm-proxy:8085/wd/hub `
     -e PROVIDER_BASE_URL=http://app-tienda:3000 `
     tienda-tests gradle --no-daemon --rerun-tasks -DbaseUrl=http://app-tienda:3000 e2e
   ```

   Tras romper el HTML (paso 3-4), para re-verificar: vuelve a `podman cp app-tienda/public/index.html ...` no aplica (el HTML vive en app-tienda, no en tests). Solo reconstruye/recrea app-tienda y vuelve a correr el `podman exec ... e2e` de arriba con `--rerun-tasks`.

   ✓*Verifica (tests verdes):* al final del log → `3 tests completed, 0 failed` y `BUILD SUCCESSFUL`. **Verificado:** 8 huellas aprendidas, incluida `grid-productos`.

   ✓*Verifica (aprendio las huellas):* `podman exec postgres-hlm psql -U healenium_user -d healenium -c "SELECT count(*) FROM healenium.selector;"` → **7** (o >0). Esta es la prueba de que guardo el DOM de referencia.

   ✓*Verifica (aun NO ha sanado):* misma consulta sobre `healenium.healing_result` → `count = 0`. Todavia no rompimos nada, no deberia haber healing.

   ✕ Si da `count = 0` en `selector`: Serenity no esta pasando por el hlm-proxy. Revisa `serenity.conf` (webdriver remoto → hlm-proxy) antes de seguir.

3 **Rompe el HTML de la app:** en `app-tienda/public/index.html` , en el `<select>` del filtro, cambia `data-testid="filtro-categoria"` por `data-testid="selector-categoria"` . (El `id` y la estructura quedan igual — solo cambia el atributo, como un cambio real de un dev).

✓*Verifica (el cambio quedo escrito):* `Select-String -Path app-tienda/public/index.html -Pattern "selector-categoria"` → debe encontrar 1 coincidencia. Y "filtro-categoria" como `data-testid` ya no debe aparecer.

4 **Reconstruye la imagen y RECREA el contenedor de la app:**

Recrear app-tienda (desde tienda-serenity-java/)

```
# 1) reconstruye la imagen con el HTML roto
podman compose build app-tienda
# 2) RECREA el contenedor (no basta 'compose up': reusa el viejo)
#    hay que borrar primero 'tests' porque DEPENDE de app-tienda (depends_on)
podman rm -f tests
podman rm -f app-tienda
podman compose up -d app-tienda
```

Ojo — por que no basta `podman compose up -d app-tienda` : si el contenedor `app-tienda` ya existe, compose lo **reusa** y sigue sirviendo el HTML viejo (lo veras como `Up 15 minutes` en vez de `Up 10 seconds`). Ademas `app-tienda` tiene un contenedor dependiente (`tests` , por `depends_on`): Podman NO deja borrar app-tienda sin borrar antes `tests` . Por eso se borran los dos y se recrea.

✓*Verifica (la app sirve el HTML roto):* `podman exec app-tienda wget -qO- http://localhost:3000/ | Select-String "selector-categoria"` → debe aparecer. Confirma que el contenedor recreado sirve el DOM roto, no el viejo. (Si usas `curl` / `Invoke-WebRequest` en PowerShell, agrega `-UseBasicParsing` para que no se cuelgue pidiendo confirmacion.)

✓*Verifica (contenedor recien recreado):* `podman ps --filter name=app-tienda` → el `Status` debe decir pocos segundos (`Up X seconds`), no minutos.

5 **Corrida con el cambio (Healenium SANA):** mismo comando del paso 2 (el test busca el selector viejo y Healenium sana):

E2E — corrida con el cambio

```
podman run --rm --network tienda-serenity-java_default `
  -e HLM_PROXY_URL=http://hlm-proxy:8085/wd/hub `
  -e PROVIDER_BASE_URL=http://app-tienda:3000 `
  tienda-tests gradle --no-daemon --rerun-tasks -DbaseUrl=http://app-tienda:3000 e2e
```

Critico — `--rerun-tasks` : sin esta bandera Gradle ve los mismos inputs del paso 2, marca la tarea `e2e` como `UP-TO-DATE` y **NO vuelve a abrir el navegador** → no hay intento de busqueda → no hay healing. Verificado: sin `--rerun-tasks` , `healing_result` quedaba en 0 filas aunque el test figurara "verde".

✓*Verifica (sigue verde pese al selector roto):* `3 tests completed, 0 failed` y `4 actionable tasks: 4 executed` (no "up-to-date"). El test busca `filtro-categoria` (que ya no existe) y aun asi pasa.

✓*Verifica (quedo registro de sanacion):* `podman exec postgres-hlm psql -U healenium_user -d healenium -c "SELECT (locator::json->>'value'), score, success_healing FROM healenium.healing_result;"` → una fila con `select#filtro-categoria` (el **nuevo** localizador con el que sano), `score ≈ 0.96` , `success_healing = t` . Para ver **original** → **nuevo** usa la consulta con JOIN del bloque "Donde ver el informe" mas abajo.

Importante — escritura asincrona: Healenium guarda la fila en `healing_result` con un pequeno **retardo** tras terminar la corrida. Si consultas de inmediato puede dar `0 rows` aunque el healing SI ocurrio. Espera unos segundos y vuelve a consultar. Verificado: la fila `filtro-categoria → select#filtro-categoria, 0.964, t` aparece al re-consultar.

✓*Verifica (screenshot generado):* `Get-ChildItem screenshots -Recurse -File` → un `.png` (~100 KB) en `screenshots/<hash>/` . Abrelo: la pagina con el `<select>` recuadrado en rojo.

✓*Verifica (dashboard web):* abre `http://localhost:7878/healenium/report` en el navegador → el reporte "SHA" muestra la sanacion. Ojo: la ruta es `/healenium/report` , NO la raiz, y solo tiene datos despues de sanar.

La prueba de que fue healing y no suerte: verde (paso 2) sin fila en `healing_result` y sin screenshot; verde (paso 5) *con* fila *y con* screenshot. La diferencia es exactamente la sanacion.

6 **Revierte y limpia:** vuelve el HTML (`selector-categoria` → `filtro-categoria`), reconstruye la app y baja el stack: `podman compose down -v` .

✓*Verifica (todo abajo):* `podman compose ps` → sin servicios. El `-v` borra tambien el volumen de PostgreSQL, asi la proxima corrida arranca sin huellas (laboratorio limpio).

Donde ver el informe: QUE identificador se ajusto (original → nuevo)

Healenium guarda cada sanacion en PostgreSQL, repartida en 2 tablas: `selector` (el localizador **original** que buscaba el test) y `healing_result` (el localizador **nuevo** con el que sano, su score y si tuvo exito). Para ver el **antes → despues** hay que cruzarlas con un JOIN:

Informe de sanacion: original vs nuevo localizador

```
podman exec postgres-hlm psql -U healenium_user -d healenium -c "
  SELECT (s.locator::json->>'value')  AS original_que_buscaba_el_test,
         (hr.locator::json->>'value') AS nuevo_con_el_que_sano,
         hr.score, hr.success_healing
  FROM healenium.healing_result hr
  JOIN healenium.healing  h ON hr.healing_id  = h.uid
  JOIN healenium.selector s ON h.selector_id  = s.uid
  ORDER BY hr.create_date DESC;"
```

Resultado real verificado

original (lo que buscaba el test)	nuevo (con el que sano)	score	exito
`[data-testid='filtro-categoria']`	`select#filtro-categoria`	0.968	true
`[data-testid='grid-productos']`	`div#grid`	0.981	true

Lee la primera fila asi: el test buscaba `[data-testid='filtro-categoria']` (que ya no existia tras romper el HTML); Healenium lo re-encontro por estructura DOM (el `id` no cambio) y **lo sano usando** `select#filtro-categoria` con 96.8% de similitud. **Ese es el identificador ajustado.** El `SCORE_CAP` es 0.6: por debajo de 60% NO sanaria (evita falsos positivos).

💡 Version corta si solo quieres el localizador nuevo y el score (sin el JOIN): `SELECT (locator::json->>'value'), score, success_healing FROM healenium.healing_result ORDER BY create_date DESC;` — ojo: ese `locator` es el **nuevo**, no el original.

Las 4 formas de ver la evidencia (version open-source)

- **1. Tabla `healing_result` en PostgreSQL** (la consulta de arriba): es la fuente de verdad. Dice localizador sanado, score y si tuvo exito.
- **2. Screenshot de Healenium (VERIFICADO):** el backend monta `./screenshots:/screenshots` (igual que el docker-compose oficial de Healenium) y **genera un PNG por cada sanacion**. Ruta real: `screenshots/<hash-del-selector>/screenshot_<fecha>.png` . La captura muestra la pagina con el elemento sanado **recuadrado en rojo** (en nuestra corrida: el `<select>` de categoria). Es la evidencia visual del antes/despues.
- **3. Dashboard web del backend (VERIFICADO):** `hlm-backend` trae una UI de reportes llamada "SHA — Self-Healing Automation" en http://localhost:7878/healenium/report . **Ojo:** es la ruta `/healenium/report` , no la raiz `/` (la raiz carga pero sin datos). Y solo muestra filas cuando ya hubo una sanacion con datos: si corres y aparece vacio, es que aun no se registro ningun healing con contenido.
- **4. Reporte Serenity:** el E2E genera su propio reporte HTML en `target/site/serenity/` , donde el escenario aparece en verde aunque el selector habia cambiado.

⚠️ **Correccion honesta:** el dashboard web de reportes del backend (puerto 7878, ruta `/healenium/report`) **SI es open-source** y viene con `hlm-backend` . Lo que es de pago (Healenium Pro) es el producto comercial aparte con mas analitica; para este curso, el reporte del 7878 + la tabla SQL + los screenshots son suficientes.

⚠️ **Nota honesta para la demo en Windows/WSL — el REGISTRO del healing es intermitente:** lo que ocurre **siempre** (100% reproducible) es lo esencial: rompes el `data-testid` de la app y el **test sigue verde** — eso ES el self-healing. Lo **intermitente** es que la fila quede escrita en `healing_result` : en Podman sobre Windows, la conexion del WebDriver remoto por el `hlm-proxy` es inestable (a veces pasa por el proxy y registra, a veces va directo al grid y no). No es el codigo ni la config. **Para clase:** (1) muestra el test verde pese al cambio — ese es el mensaje; (2) para el registro, usa la evidencia ya capturada (tabla `0.964` + screenshot en `evidencia-healing/` + dashboard 7878); (3) si lo quieres en vivo, con el stack caliente corre el E2E 2-3 veces y recuerda que `healing_result` se escribe **asincrono** (espera ~15-20 s y re-consulta).

PARTE C

Contract Testing con Pact — Python y Java

Validar que la API de la Tienda cumple el contrato que otro servicio espera, sin E2E pesados. Dos implementaciones: Python (liviana, conceptualmente clara) y Java con Pact JVM (produccion-ready).

Secciones 9-11 · Pact Python verificado (1 interaction, 0 failures) · Pact Java verificado (tests=1, failures=0)

9. Que es contract testing

El problema de los microservicios que se hablan entre si

El escenario

Tienes la API de la Tienda (**provider**) y un servicio de Notificaciones (**consumer**) que la consume. Si el equipo de la Tienda renombra `precio` a `valor`, Notificaciones se rompe en produccion... y nadie se entero hasta que exploto. ¿Como lo atrapamos antes?

E2E completo

Lento, fragil, necesita TODOS los servicios arriba a la vez. No escala cuando hay docenas de pares consumer/provider.

Mocks sueltos

Rapidos pero mienten: el mock puede quedar desactualizado respecto al provider real y nadie se da cuenta.

Contract testing

El consumer declara que espera (el contrato). El provider se verifica contra ESE contrato. Rapido Y fiel a la realidad.

La idea central de Pact (consumer-driven)

El contrato lo escribe **quien consume**, no quien provee. Notificaciones dice "yo necesito que `GET /api/productos/1` me devuelva id, nombre y precio". Pact valida la **forma** (tipos), no los valores exactos. Asi un cambio de datos NO rompe el contrato; solo lo rompe un cambio de estructura.

El ciclo en dos lados (asi funciona de verdad)

El contract testing tiene **dos fases** que ocurren en equipos y momentos distintos, sin que consumer y provider esten arriba a la vez:

- **1. Lado consumer — genera el contrato:** el test del consumer corre contra un *mock* que Pact levanta. Si el test pasa, Pact escribe el **pact file** (un JSON con cada interaccion: peticion esperada + respuesta esperada).
- **2. Lado provider — verifica el contrato:** Pact toma ese pact file y **repite cada peticion contra el provider real**. Si la respuesta real cumple la forma declarada, pasa. Si el provider cambio la estructura, **falla aqui** — antes de produccion.

La magia: el consumer no necesita al provider real para generar su contrato, y el provider no necesita al consumer para verificarlo. Solo comparten el **pact file**.

Terminos que vas a oir

Termino	Que es
Consumer	Quien llama a la API (aqui: Notificaciones). Define el contrato.
Provider	Quien expone la API (aqui: la Tienda). Se verifica contra el contrato.
Pact file	El contrato: un JSON versionable con las interacciones esperadas.
Matcher (`Like` , type)	Regla que valida la FORMA (tipo) y no el valor exacto.
Pact Broker	Servidor opcional que comparte pact files entre equipos y versiona contratos. En clase compartimos el JSON por volumen; en produccion seria el broker.

Cuando SI y cuando NO

- **Usalo cuando:** tienes servicios que se hablan por HTTP/JSON y equipos distintos los mantienen; quieres atrapar rupturas de API sin levantar todo el sistema; el numero de pares consumer/provider crece.

- **No lo necesitas cuando:** validas UN servicio aislado (eso es un test de API normal), o pruebas reglas de negocio internas (eso es test unitario). Contract testing valida el **acuerdo entre dos servicios**, nada mas.
- **No reemplaza:** ni a los tests unitarios, ni al E2E. Es una capa intermedia: mas rapida y fiable que un E2E completo para el problema especifico de "¿seguimos hablando el mismo idioma?".

📁 **¿Desde qué carpeta? (hay varias subcarpetas en `dia6/proyecto/`)**

Dentro de `dia6/proyecto/` hay 5 subcarpetas (`consumer-notificaciones`, `provider-tienda`, `provider-verify`, `pacts`, `tienda-serenity-java`). La duda típica: ¿me paro en `proyecto/` o entro a una subcarpeta? Respuesta:

Vía	Párate EN...	¿Entrar a subcarpetas?
Pact Python (sección 10)	`dia6/proyecto/` (la raíz)	**NO.** Te quedas en `proyecto/`; los comandos apuntan a las subcarpetas con `./consumer-notificaciones`, `./provider-tienda`, etc.
Pact Java (sección 11)	`dia6/proyecto/tienda-serenity-java/`	**SÍ.** Esta vía sí trabaja dentro de esa subcarpeta (`build.gradle`, `Dockerfile.tests`, `src/`).

Por qué Python se corre en la raíz: el volumen `-v "${PWD}/pacts:/pacts"` necesita que `pacts/` esté dentro de tu carpeta actual, y las imágenes se construyen con `./consumer-notificaciones` (ruta relativa a la raíz). Si entras a una subcarpeta, esos `./...` y el `${PWD}/pacts` dejan de resolver. Regla rápida: comando con `./consumer-notificaciones` → raíz (Python); comando con `tienda-tests gradle ... Pact` → `tienda-serenity-java/` (Java). PowerShell usa `${PWD}`; bash usa `$PWD`.

10. Pact en Python — implementacion liviana

Consumer en Python → contrato → provider Node verificado

Lado consumer — declara que espera

test_consumer_pact.py (fragmento)

```
pact = Consumer("notificaciones").has_pact_with(Provider("tienda-api"), ...)

cuerpo_esperado = {
    "id": Like(1), "nombre": Like("Tasty Concrete Chair"),
    "categoria": Like("hogar"), "precio": Like(106.25), "stock": Like(5),
}
(pact
    .given("existe el producto 1")
    .upon_receiving("una peticion del producto 1")
    .with_request("get", "/api/productos/1")
    .will_respond_with(200, body=cuerpo_esperado))

with pact:
    resumen = cliente.resumen_para_notificar(1)   # cliente real contra el mock
    assert "Producto" in resumen and "$" in resumen
```

`Like(...)` es la clave: Pact valida que `precio` sea un numero (del tipo del ejemplo), no que valga exactamente 106.25. El contrato es sobre la **forma**, no sobre los datos.

El flujo orquestado

Flujo Pact Python — EJECUTAR DESDE dia6/proyecto/ (la raíz, NO desde tienda-serenity-java/)

```
# Ubicate en la raiz del proyecto del dia:
#   cd dia6/proyecto
# (PowerShell usa ${PWD} en los volumenes; bash/Linux usa $PWD)

# 0) Red para que los contenedores se vean por nombre
podman network create pact-net
```

```
# 1) Construir y correr el CONSUMER: genera el contrato en ./pacts/
podman build -t consumer-notificaciones ./consumer-notificaciones
podman run --rm -v "${PWD}/pacts:/pacts" -e PACT_DIR=/pacts consumer-notificaciones

# 2) Construir y levantar el PROVIDER real en la red
podman build -t provider-tienda ./provider-tienda
podman run -d --name provider-tienda --network pact-net provider-tienda

# 3) Construir y correr el VERIFICADOR contra el provider real
podman build -t provider-verify ./provider-verify
podman run --rm --network pact-net -v "${PWD}/pacts:/pacts" `
  -e PROVIDER_URL=http://provider-tienda:3000 `
  -e PACT_FILE=/pacts/notificaciones-tienda-api.json provider-verify

# 4) Limpieza
podman rm -f provider-tienda; podman network rm pact-net
```

Resultado: "1 interaction, 0 failures — el provider CUMPLE el contrato" ✓

11. Pact en Java (Pact JVM) — produccion-ready

Misma interaccion, mas control: matchers precisos, provider states, Gradle tasks

Por que Java ademas de Python

- **Pact JVM** es la implementacion de referencia en el mundo enterprise Java/Spring Boot.
- Los matchers son mas precisos: `integerType`, `decimalType`, `stringType` — mas expresivos que `Like`.
- Se integra directamente en el build de Gradle como tareas separadas: `consumerPact` (genera) y `providerPact` (verifica).
- Mismo contrato, misma app: el `provider-tienda` (Node) es el provider de ambos.

Consumer Java — matchers mas expresivos

NotificacionesConsumerPactTest.java (fragmento)

```
@Pact(consumer = "notificaciones", provider = "tienda-api")
public RequestResponsePact productoUno(PactDslWithProvider builder) {
    PactDslJsonBody cuerpo = new PactDslJsonBody()
        .integerType("id", 1)
        .stringType("nombre", "Tasty Concrete Chair")
        .stringType("categoria", "hogar")
        .decimalType("precio", 106.25)
        .integerType("stock", 5);

    return builder
        .given("existe el producto 1")
        .uponReceiving("una peticion del producto 1")
        .path("/api/productos/1").method("GET")
        .willRespondWith().status(200).body(cuerpo)
        .toPact();
}
```

Tareas Gradle — qué hace cada una

El contract testing Java son **dos tareas que van en pareja**, y la clave es que **comparten el contrato** (el pact file). Qué hace cada una:

- `consumerPact` — corre el test del consumer contra un mock y **GENERA el contrato**: escribe `build/pacts/notificaciones-tienda-api.json`. Salida esperada: `1 test completed` + `BUILD SUCCESSFUL`. No valida nada contra la app real todavía; solo produce el JSON.

- `providerPact` — toma ese JSON y **VERIFICA** que la app real (app-tienda) cumple cada interacción. Salida esperada: `BUILD SUCCESSFUL` (contrato cumplido) o `BUILD FAILED` con el detalle del campo que no coincide.

Por qué necesitan un volumen compartido: cada `podman run --rm` es un contenedor nuevo y efímero. Si no compartes el contrato, `consumerPact` lo escribe en SU contenedor y al terminar se borra; luego `providerPact` arranca en otro contenedor vacío y falla con `NoPactsFoundException` ("no encontré ningún contrato que verificar"). El volumen `pact-data` hace que el JSON sobreviva del primero al segundo.

EJECUTAR DESDE dia6/proyecto/tienda-serenity-java/ · stack arriba · imagen tienda-tests construida

```
# 1) CONSUMER genera el contrato y lo guarda en el volumen pact-data
podman run --rm --network tienda-serenity-java_default `
  -v pact-data:/work/build/pacts `
  -e PROVIDER_BASE_URL=http://app-tienda:3000 `
  tienda-tests gradle --no-daemon consumerPact

# ver el contrato generado (opcional):
podman run --rm -v pact-data:/pacts alpine cat /pacts/notificaciones-tienda-api.json

# 2) PROVIDER lee ESE MISMO contrato (mismo volumen) y lo verifica
podman run --rm --network tienda-serenity-java_default `
  -v pact-data:/work/build/pacts `
  -e PROVIDER_BASE_URL=http://app-tienda:3000 `
  tienda-tests gradle --no-daemon providerPact
```

⚠️ **Dos claves en Windows:** (1) usa `podman run ... tienda-tests`, NO `podman compose run tests`. (2) monta `-v pact-data:/work/build/pacts` en **ambos** comandos; si lo omites, `providerPact` falla con `NoPactsFoundException` porque el contrato que genero el consumer se perdio al cerrar su contenedor.

Resultado verificado: "tests=1, failures=0, BUILD SUCCESSFUL" ✓

El provider (Node) cumple el contrato declarado por el consumer (Java). El contrato generado usa `pactSpecification 3.0.0` con matchers por tipo.

📝 **EJERCICIO PRACTICO**

Ejercicio 3 — Romper y reparar el contrato

Dos vias independientes: **3a (Python)** y **3b (Java)**. Cada paso trae su comando y la salida esperada. La idea es la misma: el contrato atrapa una ruptura de API *antes* de produccion.

Via 3a — Pact Python (desde `dia6/proyecto/`)

1. **Contrato sano:** corre el flujo completo de la seccion 10 (red + consumer + provider + verify).
   ✓ *Verifica:* el verify imprime `1 interaction, 0 failures` y "el provider CUMPLE el contrato".

2. **Rompe la API:** en `provider-tienda/servidor/server.js`, en el handler `GET /api/productos/:id`, la última línea es `res.json(p)` (devuelve el producto tal cual). Cámbiala para renombrar `precio` → `valor`:

   ```
   // ANTES (contrato cumplido):
   res.json(p);

   // DESPUES (contrato roto — renombra precio a valor):
   const { precio, ...resto } = p;
   res.json({ ...resto, valor: precio });
   ```

   ✓ *Verifica (quedó escrito):* `Select-String -Path provider-tienda/servidor/server.js -Pattern "valor: precio"` → 1 coincidencia.

   > Nota: el campo `precio` NO está escrito literalmente en el endpoint (viene dentro del objeto `p` que sale de `datos/db.json`). Por eso se rompe transformando la respuesta con `res.json(...)`, no buscando la palabra "precio" en el handler.

3. **Reconstruye el provider y re-verifica:**

   ```
   podman build -t provider-tienda ./provider-tienda
   podman rm -f provider-tienda
   podman run -d --name provider-tienda --network pact-net provider-tienda
   podman run --rm --network pact-net -v "${PWD}/pacts:/pacts" `
     -e PROVIDER_URL=http://provider-tienda:3000 `
     -e PACT_FILE=/pacts/notificaciones-tienda-api.json provider-verify
   ```

✓ *Verifica (el contrato atrapa la ruptura):* ahora el verify da `1 interaction, 1 failure` — el provider ya no devuelve `precio` .

4 **Revierte y limpia:** vuelve `valor` → `precio` , reconstruye y recrea el provider, re-verifica → vuelve a `0 failures` . Cierra: `podman rm -f provider-tienda; podman network rm pact-net` .

Via 3b — Pact Java (desde `dia6/proyecto/tienda-serenity-java/`, con `podman compose up -d app-tienda`)

1 **Contrato sano** — consumer genera el contrato y provider lo verifica. **Ojo:** el `-v pact-data:/work/build/pacts` en AMBOS comandos es lo que hace que el provider encuentre el contrato que generó el consumer (sin eso: `NoPactsFoundException`).

```
podman run --rm --network tienda-serenity-java_default -v pact-data:/work/build/pacts `
  -e PROVIDER_BASE_URL=http://app-tienda:3000 tienda-tests gradle --no-daemon consumerPact
podman run --rm --network tienda-serenity-java_default -v pact-data:/work/build/pacts `
  -e PROVIDER_BASE_URL=http://app-tienda:3000 tienda-tests gradle --no-daemon providerPact
```

✓ *Verifica:* ambas tareas `BUILD SUCCESSFUL` ; providerPact con `tests=1, failures=0` .

2 **Rompe la API:** en `app-tienda/servidor/server.js` , en el handler `GET /api/productos/:id` , cambia la última línea `res.json(p)` para renombrar `precio` → `valor` :

```
// ANTES:  res.json(p);
// DESPUES (rompe el contrato):
const { precio, ...resto } = p;
res.json({ ...resto, valor: precio });
```

> El `precio` no está escrito literal en el handler (viene del objeto `p`); por eso se rompe transformando la respuesta, no buscando la palabra "precio".

3 **Reconstruye y RECREA la app** (no basta `compose up` ; recrea el contenedor para que sirva el codigo nuevo):

```
podman compose build app-tienda
podman rm -f app-tienda
podman compose up -d app-tienda
```

✓ *Verifica:* `podman exec app-tienda wget -qO- http://localhost:3000/api/productos/1` → la respuesta trae `valor` (ya no `precio`).

4 **Re-verifica (forzando re-ejecucion):** usa el MISMO volumen `pact-data` (ahí sigue el contrato del paso 1): `podman run --rm --network tienda-serenity-java_default -v pact-data:/work/build/pacts -e PROVIDER_BASE_URL=http://app-tienda:3000 tienda-tests gradle --no-daemon --rerun-tasks providerPact` .

✓ *Verifica (atrapa la ruptura):* **BUILD FAILED**, `tests=1, failures=1` , con un error tipo "Could not find key 'precio'". Eso es el contrato haciendo su trabajo.

> Sin `--rerun-tasks` , Gradle marca `providerPact` como `UP-TO-DATE` y NO vuelve a verificar: verias un falso verde.

5 **Revierte y limpia:** vuelve `valor` → `precio` , repite el paso 3 (build + recrear) y el paso 4 → vuelve a `failures=0` . Cierra el stack: `podman compose down -v` .

12. Kubernetes (conceptual)

Donde encaja la imagen que construiste hoy

> ⚠️ **Real vs demo:** Docker/Podman, Serenity, Healenium y Pact los ejecutamos de verdad. Kubernetes lo vemos **conceptual**: para que sirve y como encajaria nuestra imagen, sin levantar un cluster en clase.

Que aporta Kubernetes

Si los contenedores son cajas, Kubernetes es el **orquestador**: decide cuantas cajas de cada servicio corren, las reinicia si mueren, las escala con la demanda y las conecta. La **misma imagen** de `app-tienda` que construimos hoy es la que K8s desplegaría.

- **Pod:** la unidad minima, envuelve uno o varios contenedores.
- **Deployment:** "quiero 3 replicas de la Tienda siempre vivas".
- **Service:** nombre estable para alcanzar esos pods.

- **readinessProbe:** K8s no envia trafico hasta que el contenedor pase el healthcheck — el mismo `/api/health` que pusimos en el Dockerfile.

Deployment de la Tienda (ilustrativo)

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: tienda }
spec:
  replicas: 3
  selector: { matchLabels: { app: tienda } }
  template:
    spec:
      containers:
        - name: tienda
          image: provider-tienda:latest   # la MISMA imagen de hoy
          ports: [{ containerPort: 3000 }]
          readinessProbe:
            httpGet: { path: /api/health, port: 3000 }
```

Contract testing en el pipeline K8s (vision completa)

En un flujo real, el contract test corre en CI **antes** de hacer deploy a K8s. Una herramienta como **Pact Broker** responde *can-i-deploy*: "¿puedo subir esta version sin romper a algun consumer?". Si la respuesta es no, el deploy a K8s se bloquea. Eso es QA integrado en el ciclo cloud-native.

Lo que construiste hoy y el puente al Dia 7

Resumen del Dia 6 completo

Capa	Herramienta	Corre de verdad
Contenedores	Docker / Podman (OCI)	Si — Podman 5.7.1
App Tienda	Node 20 + Dockerfile	Si
E2E + Screenplay	Serenity BDD + Cucumber (Java)	Si — tests=3, failures=0
Self-healing	Healenium 3.5.1 + Selenium Grid	Si — 7 selectores en PostgreSQL
Contract testing	Pact Python + Pact JVM (Java)	Si — 0 failures en ambos
Orquestacion	podman-compose (8 servicios)	Si
Kubernetes	Pod / Deployment / Service	Conceptual

Puente al Dia 7

Manana cerramos el diplomado: consolidamos todo con **metricas DORA** (frecuencia de despliegue, lead time, tasa de fallo, tiempo de recuperacion) y la **estrategia de QA cloud-native** que une los Dias 1 a 6 en una historia coherente. Cada herramienta que usamos esta semana tiene un numero DORA que mide su impacto.

Documentacion oficial:

Podman | Serenity BDD | Healenium | Pact | Kubernetes